CAGT

Recall AI
Software Architecture Document

Version 1.0

Revision History

Date	Version	Description	Author
26/Jul/26	1.0	Initial SAD for PA3: Introduction, Architectural Goals and Constraints, Use-Case Model, and Logical View (Architectural Representation, Use-Case-to-Component Mapping, AI Agents, and the Authentication / Study-Plan and Concept-Graph / Database components with class and ER diagrams). Deployment View and Implementation View left blank per template guidance (PA4 deliverable).	Nguyen The Quan

Table of Contents

Software Architecture Document

1. Introduction

Purpose: This Software Architecture Document (SAD) describes the software architecture of Recall AI, an AI-assisted study application built for the Introduction to Software Engineering course. It records the architectural decisions made about the system, gives the team a common vocabulary, and serves as the PA3 architecture deliverable.
Scope: This document covers the components implemented as of the current sprint of the Recall AI MVP: user authentication, study-plan creation with AI-assisted concept extraction, the concept-graph data model, and the underlying database. Components still under active development (AI Examiner interview engine, Focus Session, Dashboard & Visualization) are noted where their data model already exists but their service-layer implementation does not yet.
Definitions, Acronyms, and Abbreviations: SAD - Software Architecture Document. SRE - Scheduling & Remediation Engine, the system's internal deterministic scheduling/traceback module. DAG - Directed Acyclic Graph. NFR - Non-Functional Requirement. SPA - Single-Page Application. JWT - JSON Web Token. UC - Use Case. MVP - Minimum Viable Product. C4/C5/C6 - the three hard architectural constraints defined in the Software Development Plan (see Section 2).
References: Vision Document v1.0 (PA1); Software Development Plan v1.2 (PA2); Use-Case Specification v2.0 (PA3); schema.prisma (source of truth for the Database component).
Overview: Section 2 lists the architectural goals and constraints. Section 3 references the use-case model. Section 4 (Logical View) presents the 3-tier architectural representation, the use-case-to-component mapping, the AI agents, and the design of each implemented component. Sections 5 and 6 (Deployment View, Implementation View) are left for the PA4 iteration of this document, per the template's own guidance.

2. Architectural Goals and Constraints

The following goals and constraints carry over from the Vision Document and the Software Development Plan (Section 2.2); they shape every design decision described in Section 4.

- Platform: the system is a web-based Single-Page Application (SPA), built with React + TypeScript, and must run in modern desktop browsers (Chrome, Safari, Edge, Firefox).
- External dependency: the AI Study Planner and AI Examiner depend on an active connection to the external Google Gemini API; the system must degrade gracefully (see Reliability below) rather than fail outright when that dependency is unavailable.
- Cost & resource constraint: zero budget - every tool and service runs on a free tier (Gemini API free tier, free hosting, GitHub, Figma). AI usage is hard-capped to control cost and latency: a maximum of 3 question-answer turns per concept per Interview session (constraint C6).
- C4 - AI is never the orchestrator: every routing decision (when to stop a turn, when to switch concepts, when to trigger a concept traceback) is deterministic software logic, never left to the language model. The AI is invoked only for narrow, fixed-JSON-schema tasks.
- C5 - the AI Examiner must never fabricate questions outside the student's uploaded material; no external knowledge is injected into generated content.
- Reliability - AI Fallback Mechanism: if the Gemini API fails, times out, or exceeds its quota, the system degrades to a self-grading mode using previously cached questions (static flashcards) instead of failing the session.
- Data integrity: the system uses a relational database (PostgreSQL, ACID transactions) so the concept graph can never be left with orphaned concepts or corrupted prerequisite edges.
- Cycle prevention: every concept graph, whether AI-generated or student-edited, is validated as a Directed Acyclic Graph (DAG) before being saved; cyclic edges are rejected or automatically removed and logged, never silently accepted.
- Schedule: a fixed 10-week, 5-sprint schedule with no possibility of extension, and a team fixed at 5 members - architecture choices favor implementation speed and free-tier services over long-term scalability.

3. Use-Case Model

Recall AI's use-case model is organized into five modules: Account Management (AM), AI Study Planner (SP),

Focus Session (FS), AI Examiner (AE), and Dashboard & Visualization (DB), totalling 42 use cases across all planned sprints. The diagrams below, already produced for the use-case model (PA2) and still structurally valid after the Use-Case Specification v2.0 revision (PA3), show the actors and use cases per module. Full flows, pre/post-conditions, and the three use cases added in v2.0 (Dashboard Overview, Study Material Management, Session History) are detailed in the Use-Case Specification v2.0 document.

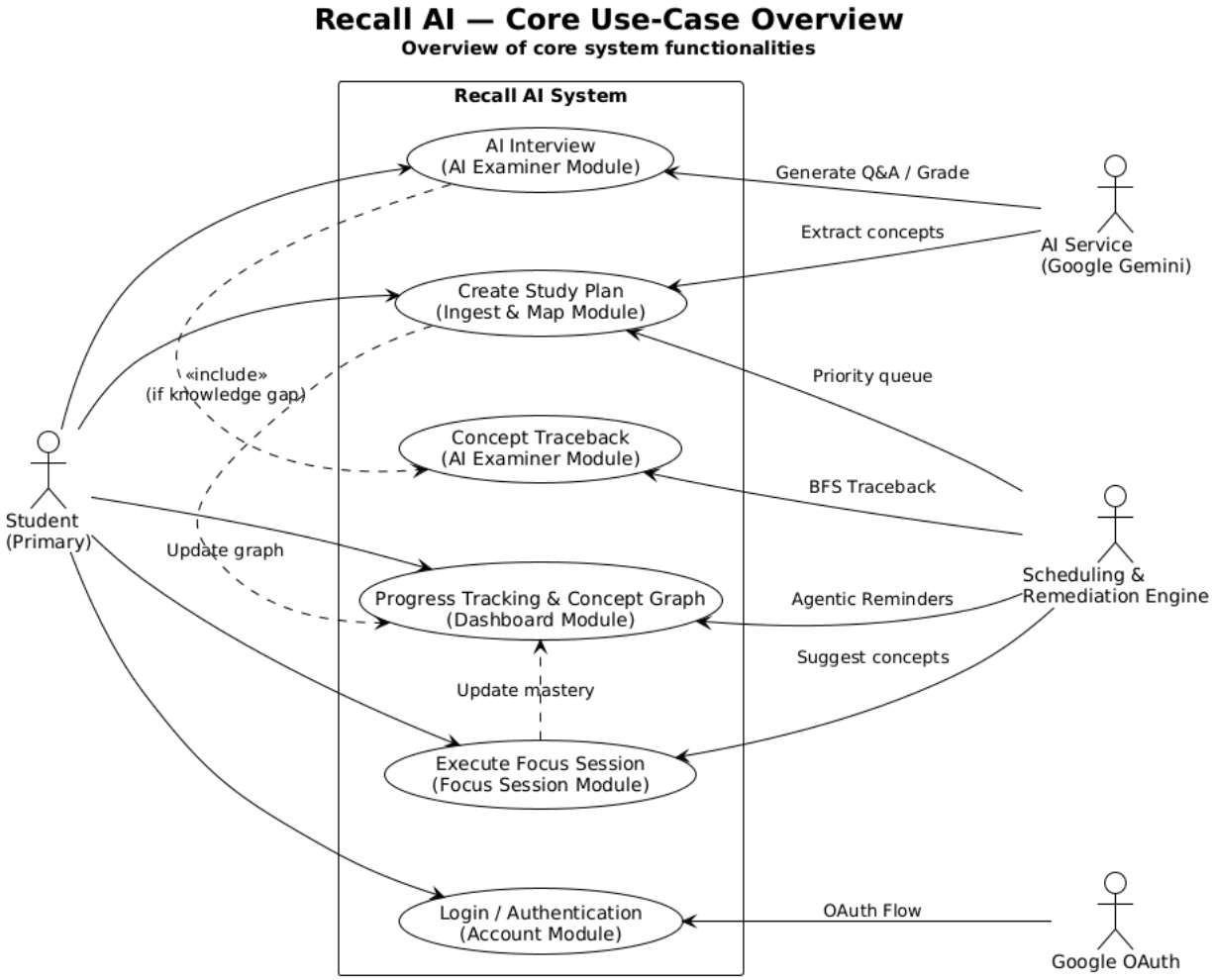

Figure 3.1 - Use-Case Overview

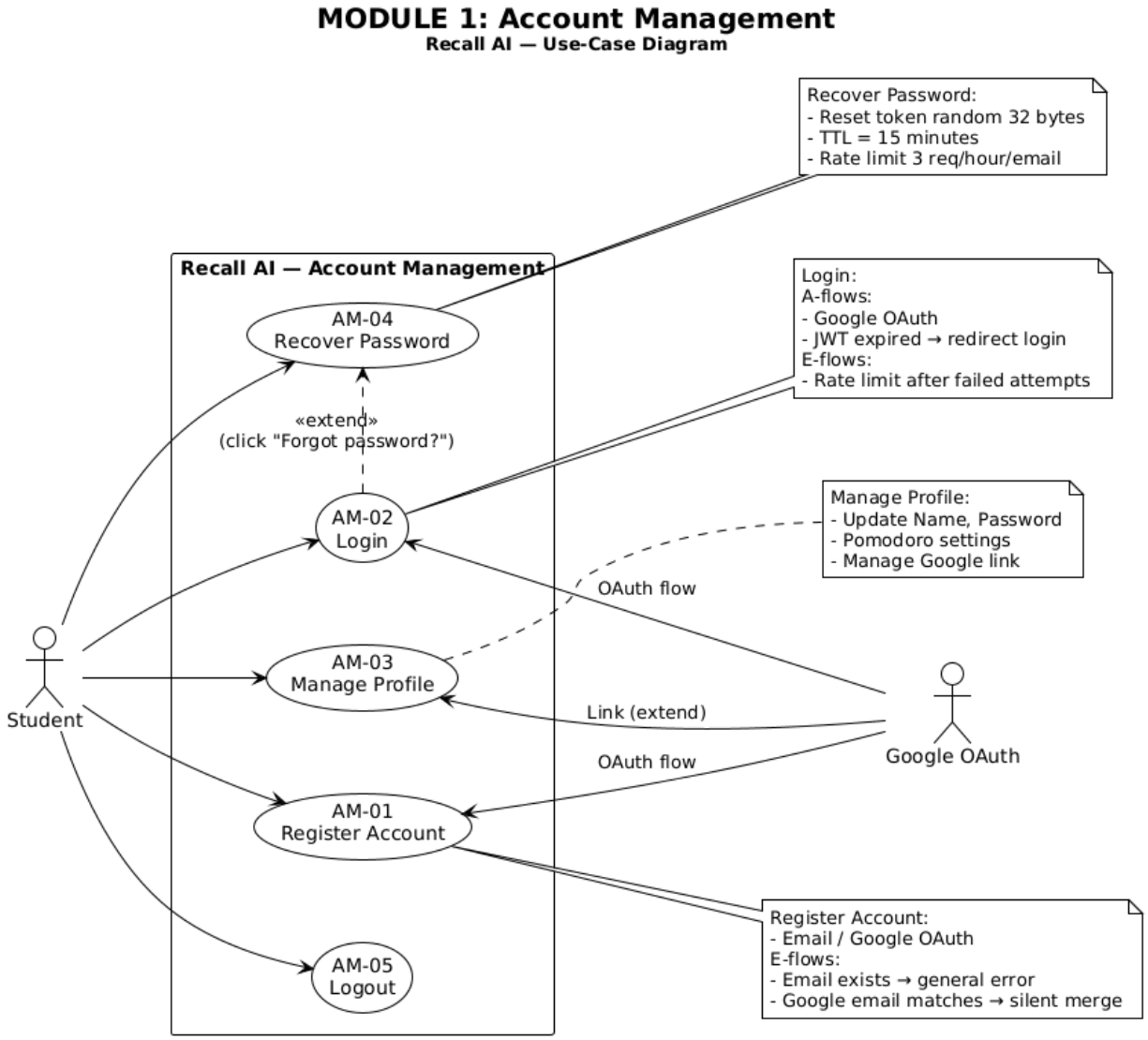

Figure 3.2 - Account Management (AM)

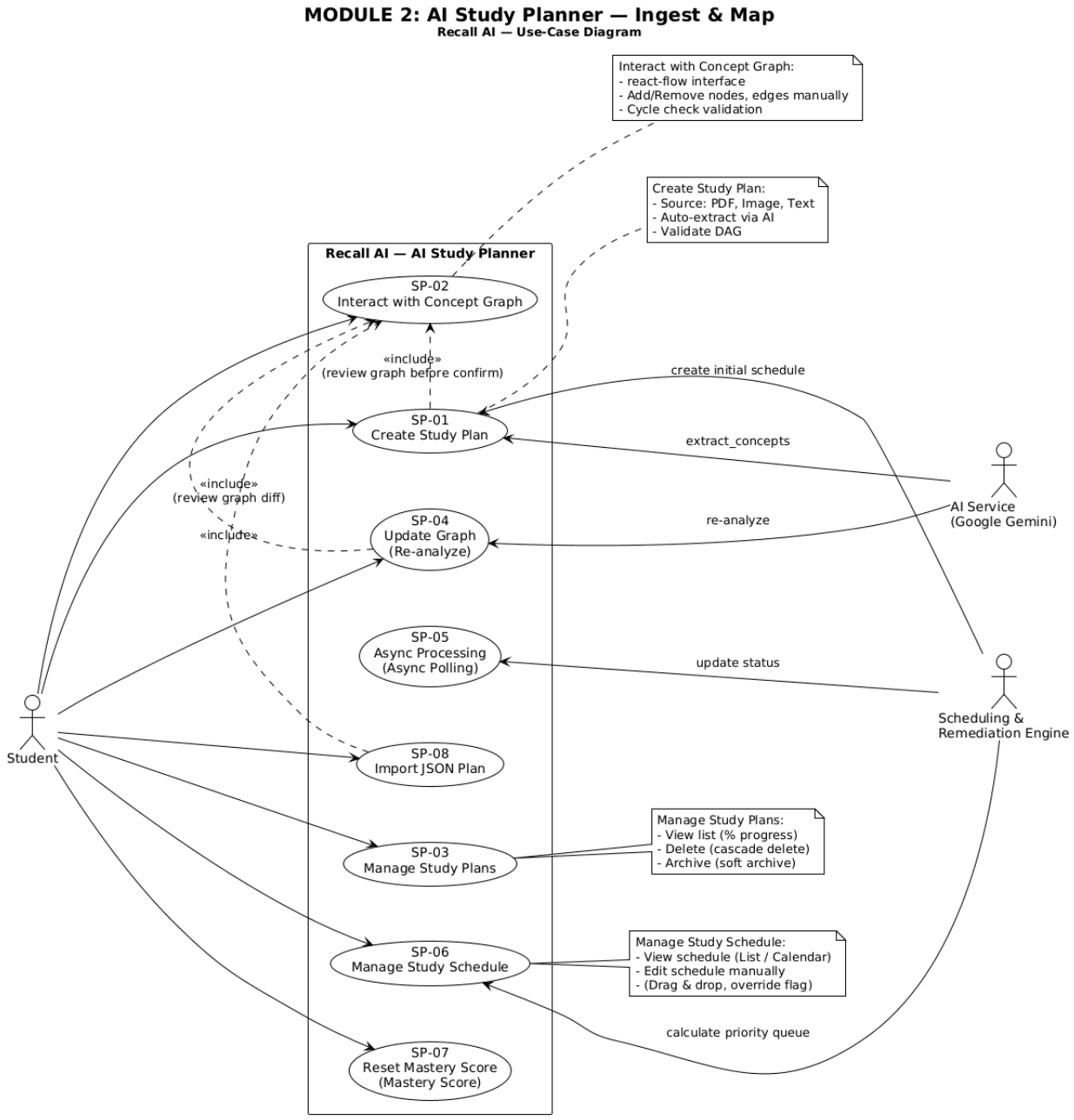

Figure 3.3 - AI Study Planner (SP)

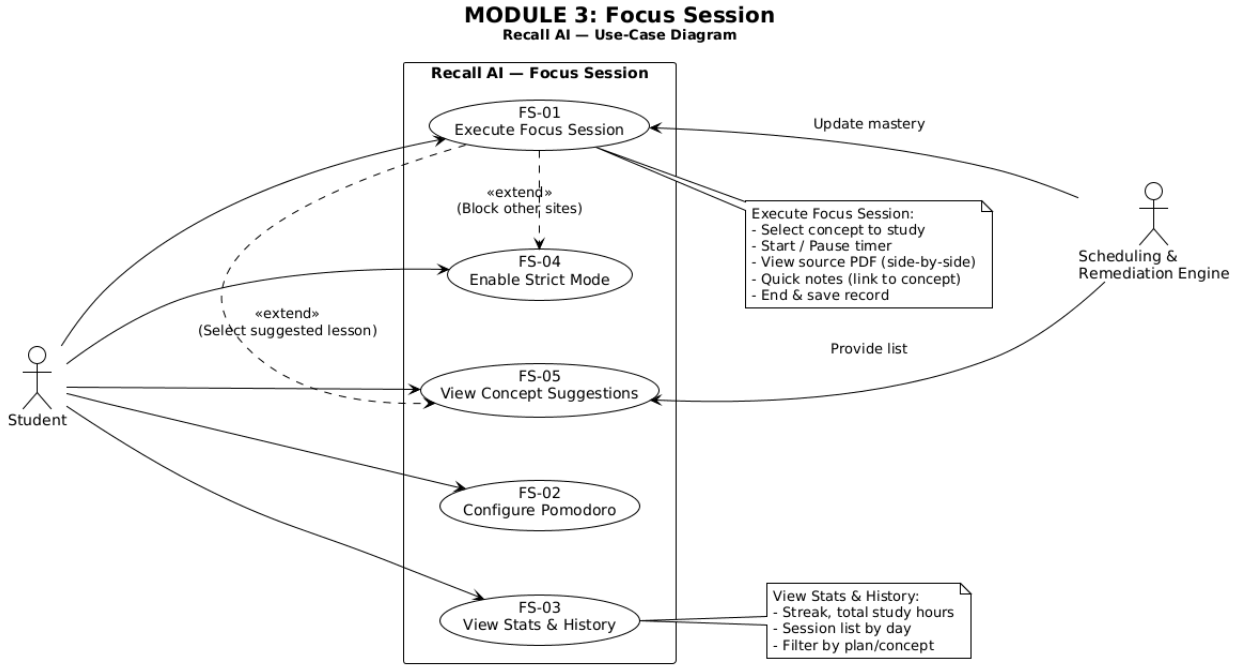

Figure 3.4 - Focus Session (FS)

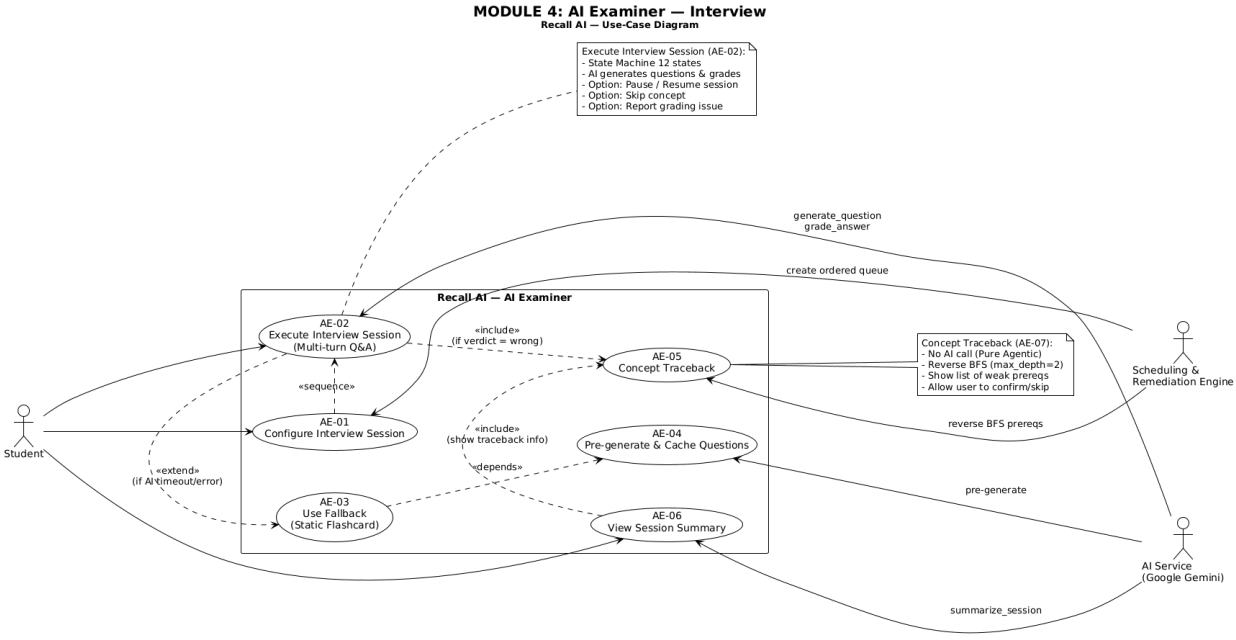

Figure 3.5 - AI Examiner (AE)

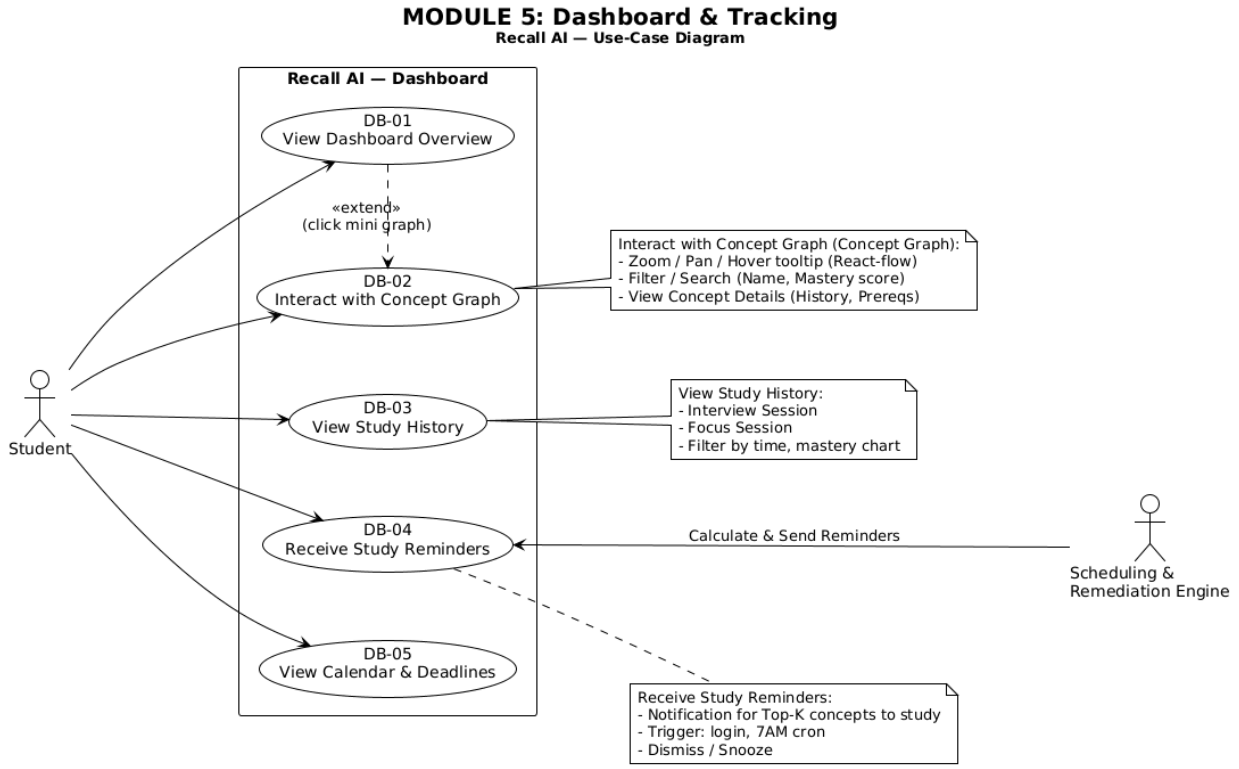

Figure 3.6 - Dashboard & Visualization (DB)

4. Logical View

4.1 Architectural Representation

Recall AI follows a 3-tier architecture: a Presentation tier (the React SPA), an Application tier (the Express.js REST API, including the internal Scheduling & Remediation Engine), and a Data tier (PostgreSQL via Prisma). The Application tier additionally depends on one external service, the Google Gemini API, for AI-assisted extraction and grading tasks.

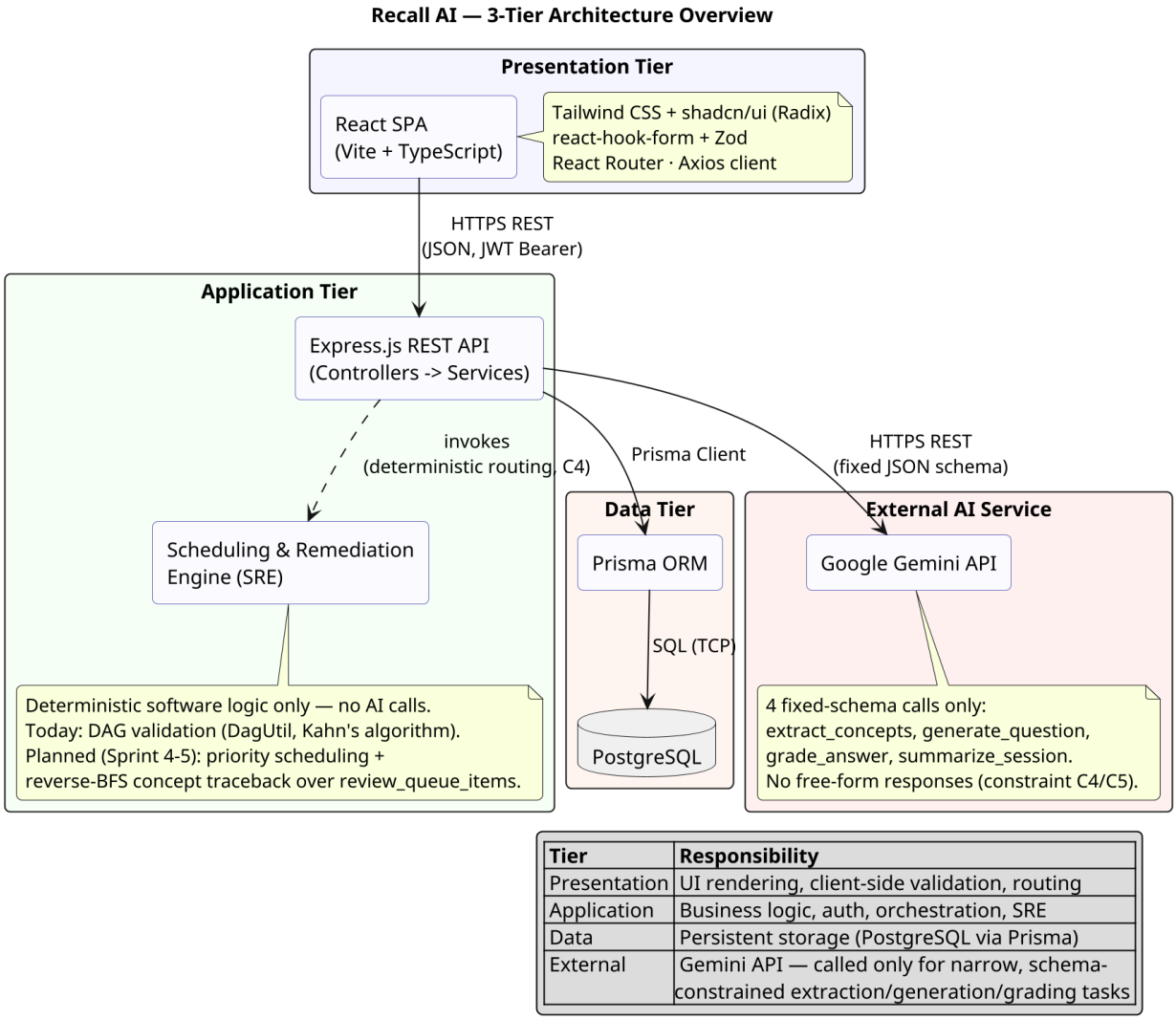

Figure 4.1 - 3-Tier Architecture Overview

The table below lists the technology used in each tier.

Tier	**Technology**
Presentation	React 19 (Vite) + TypeScript; Tailwind CSS v4 + shadcn/ui (Radix primitives); react-hook-form + Zod; React Router v7; Axios HTTP client
Application	Node.js + Express.js 5 (TypeScript); Zod request validation; JWT (jsonwebtoken) + bcryptjs; Multer file upload; Helmet + CORS
Data	PostgreSQL, accessed through Prisma ORM 7
External AI Service	Google Gemini API, via the official @google/genai SDK

Components communicate over HTTPS using REST endpoints under /api/v1, exchanging JSON payloads. The Presentation tier authenticates with a JWT Bearer token issued by the Authentication component. The Application

tier talks to the Data tier through the Prisma Client (SQL over a TCP connection) and to the external AI service over HTTPS REST with a fixed, pre-declared JSON response schema - never a free-form response.

4.2 Use-Case-to-Component Mapping

The table below maps each use-case module to the component currently responsible for it.

Use-Case Module	Responsible Component
AM-01 .. AM-05 (Account Management)	Authentication
SP-01, SP-02, SP-05, SP-06 (Create / Edit / Re-analyze Study Plan)	Study-Plan & Concept-Graph
All persisted data for every module (Users, Study Plans, Concepts, Concept Edges, Interview Sessions, Focus Sessions, Review Queue, etc.)	Database
FS-01..FS-07 (Focus Session), AE-01..AE-10 (AI Examiner), DB-01..DB-09 (Dashboard & Visualization)	Planned - Sprint 4-5 (database schema already defined; dedicated backend service/controller code not yet implemented)

4.3 AI Agents

Recall AI has two AI/agentic actors, kept strictly separate per constraint C4:
Google Gemini API (external): an external, stateless LLM service invoked only for four fixed-JSON-schema calls - extract_concepts, generate_question, grade_answer, and summarize_session (see gemini.service.ts). It never decides control flow: every call returns structured JSON validated against a Zod schema before use, and a malformed or free-form response is treated as an error, not as an instruction to follow (constraints C4, C5).
Scheduling & Remediation Engine - SRE (internal): the system's own deterministic engine - no AI calls, fully unit-testable. Today it is implemented as DagUtil.validateAndFixDag, which validates and repairs the concept graph using Kahn's algorithm. Its planned Sprint 4-5 responsibilities - priority-queue scheduling and the depth-limited reverse-BFS concept traceback - operate on the review_queue_items table already defined in the database schema.
Components communicate with both agents over REST APIs using a fixed, pre-declared JSON schema in both directions - the AI Study Planner sends a schema-constrained prompt to Gemini and validates the schema-constrained JSON it gets back; no component ever accepts or forwards a free-form natural-language response as if it were structured data.

4.4 Component: Authentication

The Authentication component implements Account Management (AM-01, AM-02, AM-04) and spans both the Application tier (server) and the Presentation tier (client).
Server: On the server, AuthController exposes four HTTP endpoints (register, login, refresh, me) and delegates all business logic to AuthService, which hashes passwords with bcrypt (10 salt rounds), issues a short-lived access token (15 minutes) and a longer-lived refresh token (7 days) via JwtUtil, and persists users through the Prisma Client. AuthMiddleware guards every protected route by verifying the Bearer token and attaching the authenticated user's id to the request. All errors are raised as the custom AppError class (extends Error) and handled centrally by errorHandler.
Note: AuthService, AuthController, AuthMiddleware, and JwtUtil are implemented as stateless functional modules rather than instantiated classes; AppError is the only class in this component in the strict OOP sense. The class diagram below reflects this honestly, using the <<module>>/<<service>> stereotype for the functional modules.

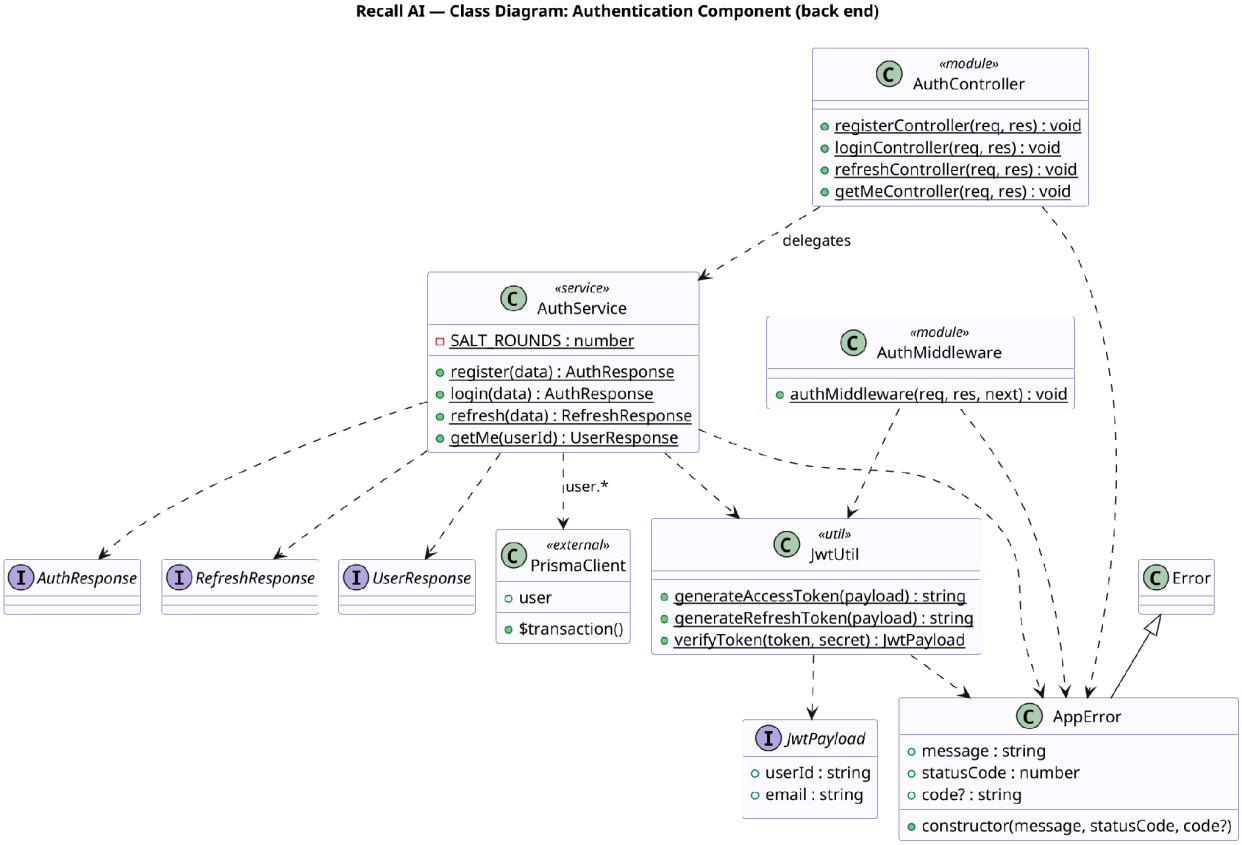

Figure 4.2 - Class Diagram: Authentication Component (Server)

Client: On the client, an AuthProvider React component owns the authentication state and exposes it through a custom useAuth hook backed by AuthContext. LoginForm and SignupForm (react-hook-form + Zod validation) call useAuth().login / .register, and a ProtectedRoute component guards authenticated routes, redirecting to /login when unauthenticated. AuthApi wraps the underlying REST calls, and a single Axios apiClient instance transparently refreshes an expired access token on a 401 response before retrying the original request.

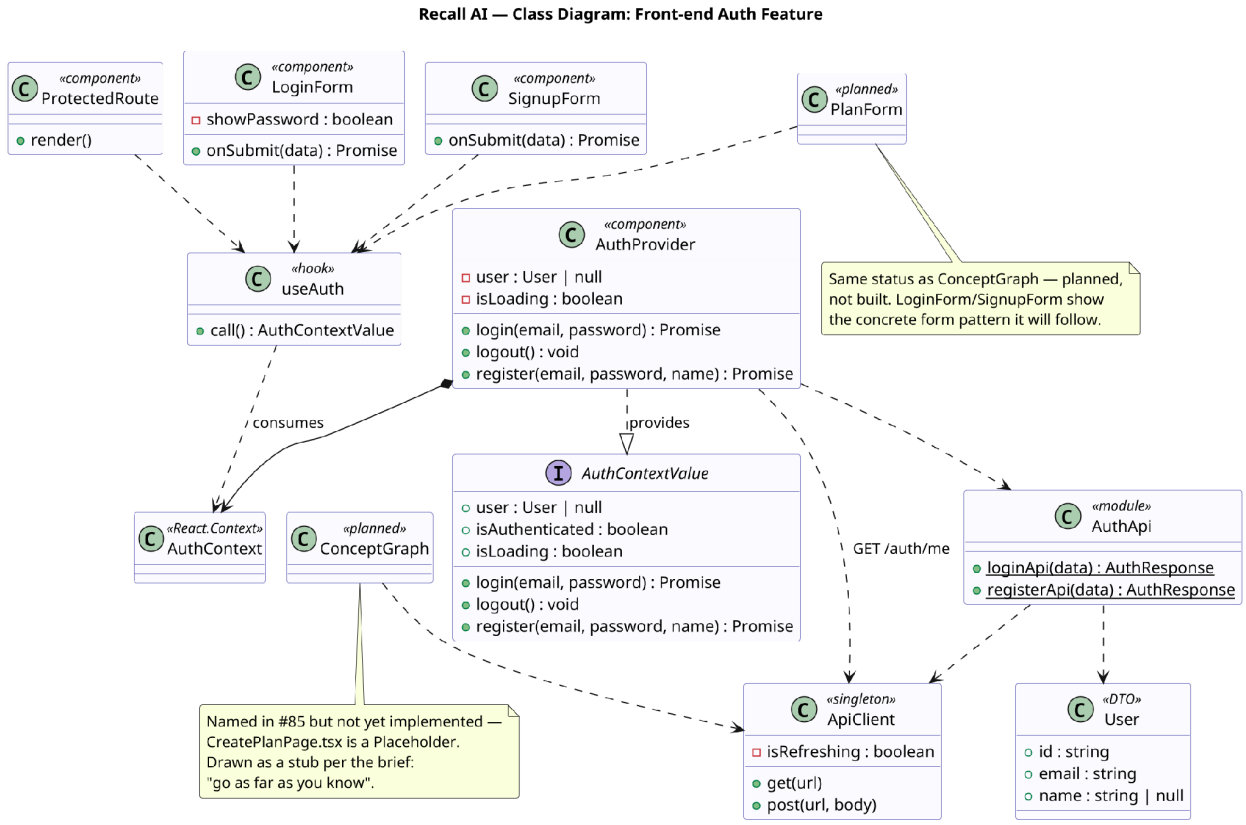

Figure 4.3 - Class Diagram: Authentication Component (Client)

4.5 Component: Study-Plan & Concept-Graph

The Study-Plan & Concept-Graph component implements study-plan creation (SP-01), graph editing (SP-02), and asynchronous document analysis (SP-06). PlanController accepts an uploaded document, stores it through the StorageService abstraction (LocalStorageService for the MVP, created by the createStorageService factory), persists the plan via PlanService, and triggers AnalysisService in the background.
AnalysisService calls GeminiService - the only class permitted to talk to the Gemini API - to extract a concept graph, then validates it as a Directed Acyclic Graph using the pure-function DagUtil.validateAndFixDag (Kahn's algorithm) before persisting Concepts and ConceptEdges. GraphService exposes the same DAG validation for the student-facing graph editor (SP-02), rejecting any student edit that would introduce a cycle rather than auto-fixing it.

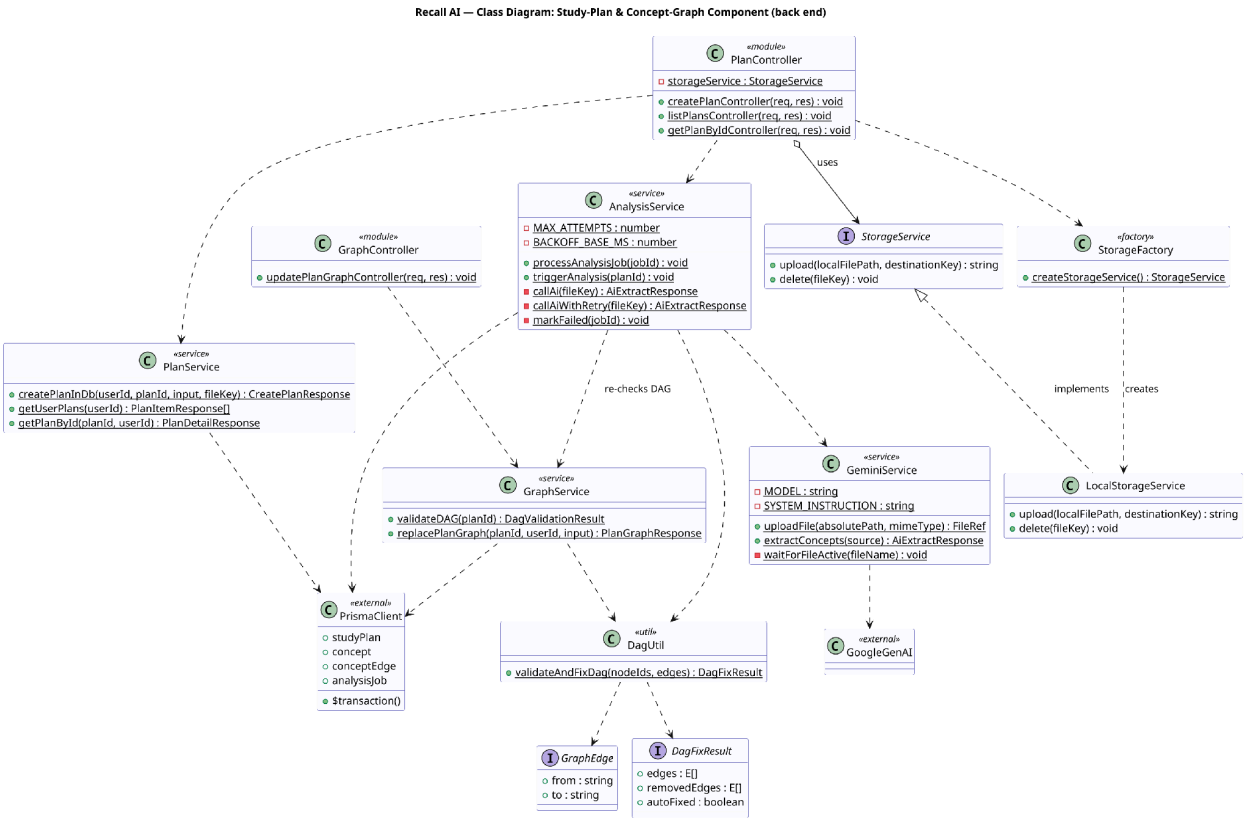

Figure 4.4 - Class Diagram: Study-Plan & Concept-Graph Component

4.6 Component: Database

The Database component persists all application state in PostgreSQL, accessed through the Prisma ORM. The schema currently defines 10 tables and 11 enumerated types: users, study_plans, concepts, and concept_edges (the core concept-graph model, where concept_edges is the associative entity resolving the many-to-many prerequisite relation between concepts); analysis_jobs and question_cache (supporting the Study-Plan component); and interview_sessions, interview_turns, focus_sessions, and review_queue_items (the Sprint 4 schema for the AI Examiner, Focus Session, and Scheduling & Remediation Engine, ahead of their service-layer implementation). Every foreign key cascades on delete, since the data forms a strict ownership tree rooted at users. A small number of references (for example review_queue_items.source_session_id) are deliberately left as soft references without a foreign-key constraint, where the referenced row's lifecycle is independent of the referencing row. See the companion Database Design document for the full entity-relationship catalogue.

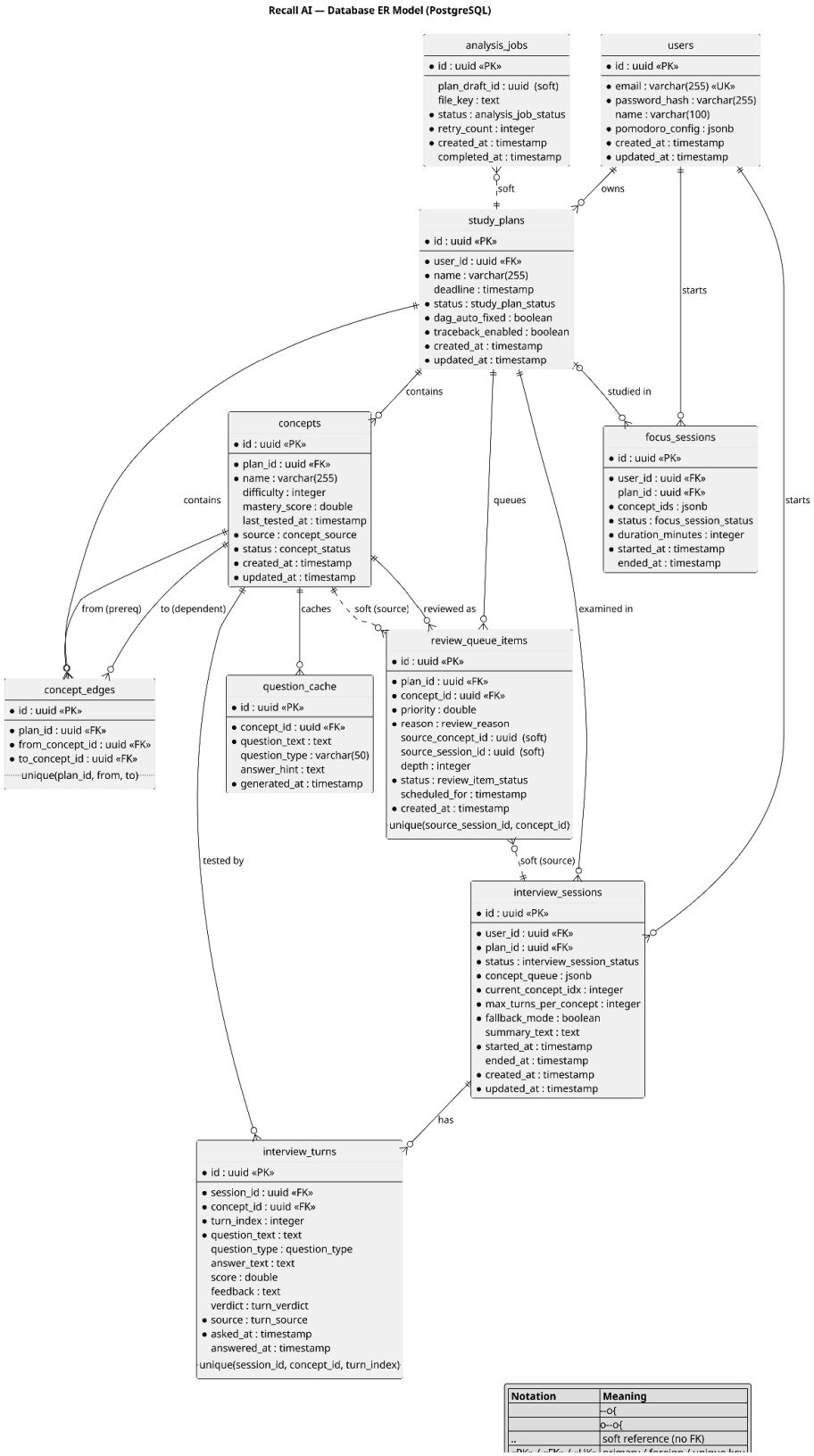
Recall AI — Database ER Model (PostgreSQL)
analysis_jobs
• id : uuid «PK»
plan_draft_id : uuid (soft)
file_key : text
• status : analysis_job_status
• retry_count : integer
• created_at : timestamp
completed_at : timestamp
users
• id : uuid «PK»
• email : varchar(255) «UK»
• password_hash : varchar(255)
name : varchar(100)
• pomodoro_config : jsonb
• created_at : timestamp
• updated_at : timestamp
soft
owns
study_plans
• id : uuid «PK»
• user_id : uuid «FK»
• name : varchar(255)
deadline : timestamp
• status : study_plan_status
• dag_auto_fixed : boolean
• traceback_enabled : boolean
• created_at : timestamp
• updated_at : timestamp
starts
contains
studied in
concepts
• id : uuid «PK»
• plan_id : uuid «FK»
• name : varchar(255)
difficulty : integer
mastery_score : double
last_tested_at : timestamp
• source : concept_source
• status : concept_status
• created_at : timestamp
• updated_at : timestamp
focus_sessions
• id : uuid «PK»
• user_id : uuid «FK»
plan_id : uuid «FK»
• concept_ids : jsonb
• status : focus_session_status
• duration_minutes : integer
• started_at : timestamp
ended_at : timestamp
contains
queues
starts
from (prereq)
to (dependent)
caches
soft (source)
reviewed as
examined in
concept_edges
• id : uuid «PK»
• plan_id : uuid «FK»
• from_concept_id : uuid «FK»
• to_concept_id : uuid «FK»
unique(plan_id, from, to)
question_cache
• id : uuid «PK»
• concept_id : uuid «FK»
• question_text : text
question_type : varchar(50)
answer_hint : text
• generated_at : timestamp
review_queue_items
• id : uuid «PK»
• plan_id : uuid «FK»
• concept_id : uuid «FK»
• priority : double
• reason : review_reason
source_concept_id : uuid (soft)
source_session_id : uuid (soft)
depth : integer
• status : review_item_status
scheduled_for : timestamp
• created_at : timestamp
unique(source_session_id, concept_id)
tested by
soft (source)
interview_sessions
• id : uuid «PK»
• user_id : uuid «FK»
• plan_id : uuid «FK»
• status : interview_session_status
• concept_queue : jsonb
• current_concept_idx : integer
• max_turns_per_concept : integer
• fallback_mode : boolean
summary_text : text
• started_at : timestamp
ended_at : timestamp
• created_at : timestamp
• updated_at : timestamp
has
interview_turns
• id : uuid «PK»
• session_id : uuid «FK»
• concept_id : uuid «FK»
• turn_index : integer
• question_text : text
question_type : question_type
answer_text : text
score : double
feedback : text
verdict : turn_verdict
• source : turn_source
• asked_at : timestamp
answered_at : timestamp
unique(session_id, concept_id, turn_index)
Notation
Meaning
–o{
o–o{
..
soft reference (no FK)

Figure 4.5 - Entity-Relationship Model: Database Component

5. Deployment

To be completed in PA4, per the template's own guidance for this section.

6. Implementation View

To be completed in PA4, per the template's own guidance for this section.